

RCA-Agent

장애 원인 분석을 자동화한 Spring AI 기반 RCA 에이전트

개인 프로젝트 · 1인 개발 · 2026. 07 ~ 08

Java 21 · Spring Boot 3.5 · Spring AI · Grafana Tempo / Loki / Mimir · Grafana Alloy · K3s · Kafka · Redis / MongoDB / MySQL

MSA 기반 서비스에 Grafana Tempo · Loki · Mimir 모니터링을 직접 구축하고, 그 위에서 장애 원인을 자동 분석하는 RCA 에이전트를 개발했습니다. 개발 과정에서 프롬프트 개선에만 머무르지 않고, 12종의 장애 상황을 직접 재현해 실험하고, 결과를 바탕으로 관측·탐색·분석 과정을 단계적으로 개선했습니다.

대표 성과

지표	결과	비고
RCA 정확도 (전 문항 평균, 회차 1 → 3)	72점 → 95점	차트 11문항 실행 평균 · 최대 개선 문항 4 → 100점
조사당 비용	33% ~ 47% 절감	분석 컨텍스트 34.5% ~ 54.5% 감소

핵심 문제 해결 사례

- **탐색 구조 개선** - 실제 장애가 분석 대상 시간 안에 있었지만, LLM 이 넓은 범위에서 조사할 시간대와 원인 후보를 직접 선택하면서 장애 구간을 놓쳤습니다. 관측 신호를 먼저 장애 사건별로 묶어 후보를 만들고, LLM 은 이 후보에서 연관 사건을 선택하도록 변경했습니다. LLM 이 처음부터 장애 구간을 찾아야 했던 방식을, 관측된 장애 사건 중에서 선택하는 방식으로 바꿔 탐색 누락을 줄였습니다. 그 결과 장애 탐지 점수를 4 점 → 85 점 → 최종 100 점까지 개선했습니다. → 본문 2)
- **관측 데이터 재구성** - 로그는 ERROR/WARN 헤더 위주로 조회돼 스택 트레이스의 실제 예외 발생 위치가 잘리고, 트레이스는 span 목록으로만 전달돼 서비스 간 호출 관계가 드러나지 않아 장애 원인을 판단할 수 있는 정보가 충분히 전달되지 않았습니다. 예외 패턴을 조회 조건에 추가해 잘린 스택 트레이스를 복원하고, span 목록은 원본을 유지한 채 서비스 간 호출 관계를 요약해 먼저 전달했습니다. 그 결과 정답 신호가 로그 스택에만 있던 문항은 53 점 → 88 점으로 올랐고, 도구 세대 전체로는 전 문항 평균 +18 점을 기록했습니다.
- **실측 기반 컨텍스트 압축** - 분석 비용이 데이터 양에 비례해 계속 커지는 문제가 있었습니다. 확인해 보니 원인 판단에 사용되지 않는 로그가 컨텍스트의 대부분을 차지하고 있었습니다. 원인 근거는 그대로 보존하고 불필요한 로그만 압축하는 방식으로 개선해, 분석 품질 저하 없이 로그 57%, 조사 비용 33~47%를 줄였습니다. → 본문 4)
- **기술 도입 전 실익 검증** - E2E Trace 실측으로 OTel Agent 전환의 실익을 확인한 뒤, 전환 대신 기존 계속 유지를 결정했습니다. → 부록

검증 방법

- **12 종 장애를 직접 발생시켜 테스트** - MongoDB 장애, Kafka DLQ 재처리 실패, 컨슈머 전체 중단, 조회 API NPE, 중복 저장 버그, 인증 서버 전체 다운, 인증 장애 중 익명 폴백, 인프라 복합 장애 등 12 가지 상황을 직접 재현했습니다.
- **12 종 모두 동일한 기준으로 평가** - 분석 성능은 단순히 원인을 맞혔는지만 평가하지 않았습니다. 실제 장애 시나리오별 정답과 만점 · 부분점 · 0 점 기준을 사전에 정의하고, 원인 적중 40 점 · 근거 타당성 25 점 · 탐색 타당성 15 점 · 영향 판정 10 점 · 잘못된 원인 지목 여부 5 점 · 조치 타당성 5 점의 6 개 항목으로 채점했습니다.

특히 원인만 맞히고 영향 범위를 잘못 판단하거나 탐색 과정에서 잘못된 대상을 선택한 경우를 별도로 구분해, LLM 이 운 좋게 정답을 맞힌 것과 실제 관측 근거를 바탕으로 RCA 를 수행한 것을 구분했습니다.

- **반복 테스트로 결과를 확인** - 각 장애를 2 회 이상 반복 테스트하고 결과의 편차를 확인했습니다. 결과가 낮게 나온 경우에는 실제 백엔드에 해당 근거가 존재했는지 직접 쿼리해, AI 의 판단 문제인지 관측 데이터의 문제인지 구분했습니다.

회차 평균은 차트에 표시된 문항별 실행 평균 점수를 그대로 평균한 값입니다.

정민채 · jmcabc0322@gmail.com

ARCHITECTURE

자연어 질문 → 장애 구간 탐색 → Trace · Log · Metric 수집 → Evidence 정제 · 구조화 → AI 분석 → 원인
과 근거 출력

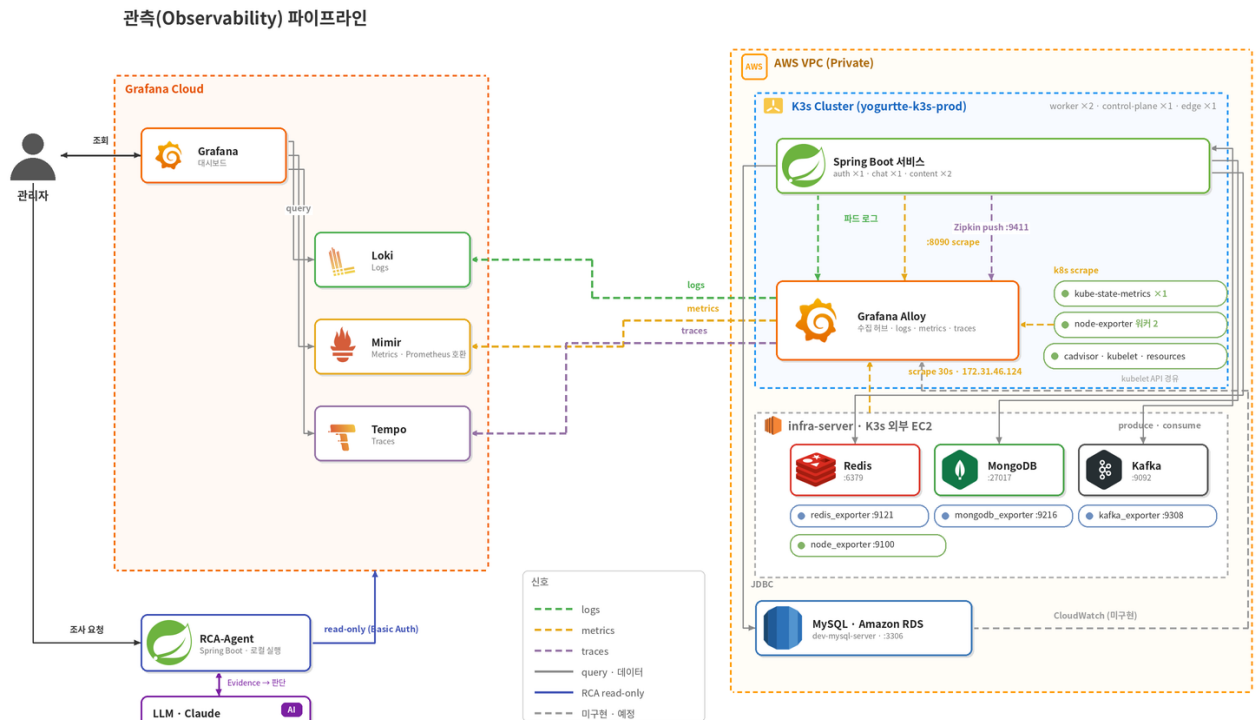


그림 1) 관측 파이프라인 구조도

조사 파이프라인은 탐색과 분석 두 단계로 설계했습니다. 사용자의 자연어 질문만으로는 장애 시간 창을 정확히 좁히기 어렵기 때문에, 탐색 단계는 넓은 창을 예러 발생률 · 지연 · 메트릭 변화 같은 집계 수치로만 훑어 장애 사건 후보를 만들고, 분석 단계는 선택된 사건의 좁힌 구간에만 트레이스 전문 · 로그 원문 같은 원본 데이터를 씁니다. 넓은 창은 싸게 훑고, 비싼 해상도는 좁힌 구간에만 쓰는 구조입니다.

1) 장애 분석 실패 원인의 단계별 검증

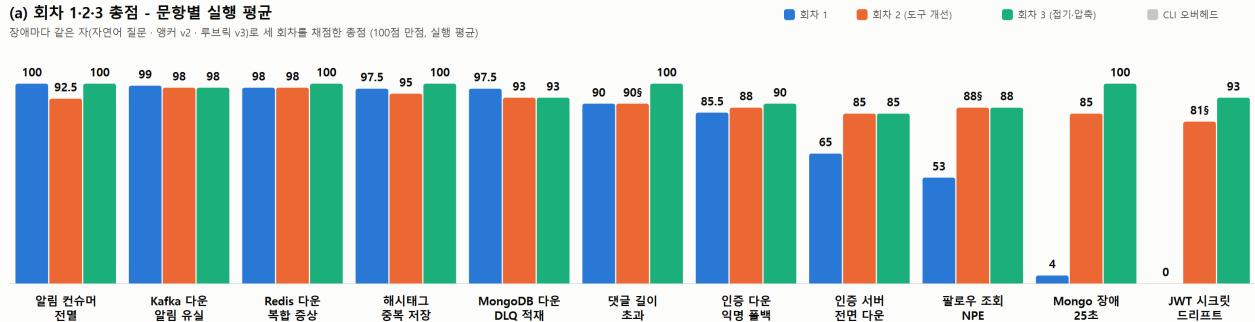
측정 결과만으로 원인이 명확하지 않을 때는 다음 개선 방향을 정할 수 없었습니다. 모델이 신호를 놓친 것인지, 수집이 부족한 것인지, 계측 자체의 한계인지 구분해야 했기 때문입니다. 그래서 원인을 관측 · 수집 · 추론 단계로 역추적하는 절차를 만들고 평가 과정에 포함했습니다.

원본 데이터가 있었나? → 쿼리가 가져왔나? → 수집·변환 과정에서 보존됐나? → 최종 Evidence에 포함됐나? → 그런데도 LLM이 놓쳤나?

2) 장애를 놓치는 원인을 추적해 탐색 구조를 개선했다

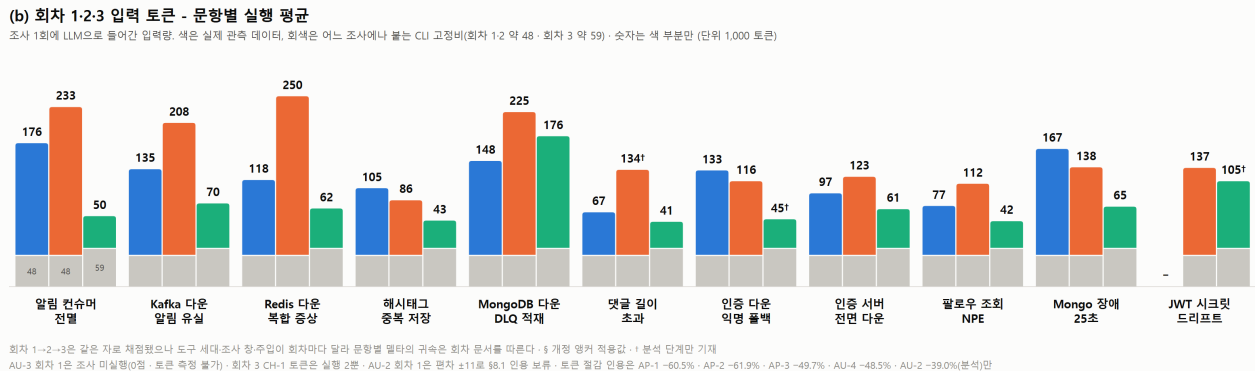
(a) 회차 1-2-3 총점 - 문항별 실행 평균

장애마다 같은 자(자면어 질문·영커 v2·후보 v3)로 세 회차를 재점한 총점 (100점 만점, 실행 평균)



(b) 회차 1-2-3 입력 토큰 - 문항별 실행 평균

조사 1회에 LLM으로 들어간 입력 토큰, 색은 실제 관측 데이터, 회색은 어느 조사에나 붙는 CU 고정비(회차 1-2 약 48 · 회차 3 약 59) · 숫자는 색 부분만 (단위 1,000 토큰)



회차 1~2~3은 같은 자료 재점했으나 도구 세대 조사 장 주입이 회차마다 달라 문항별 점수의 귀속은 회차 문서를 따릅니다. 6 개항 영커 적용값 · 1 분석 단계만 기재
AU-3 회차 1은 조사 미실행(0점 · 토큰 측정 불가) · 회차 3 CH-1 토큰은 실행 2분 · AU-2 회차 1은 편차 ±11로 \$8.1 인용 보류 · 토큰 절감 인용은 AP-1 ~60.5% · AP-2 ~61.9% · AP-3 ~49.7% · AU-4 ~48.5% · AU-2 ~39.0%(분석)만

회차별 문항 총점(위)과 입력 토큰(아래). 점수는 채널이 막혔던 문항에서만 오르고, 접기·압축을 적용한 마지막 회차에서는 컨텍스트가 절반 이하로 줄어도 점수가 내려간 문항이 없습니다.

문제

RCA 에이전트를 검증하던 중, 조사 구간에 실제 장애가 포함되어 있었으나 원인을 찾지 못하는 문항이 있었습니다. 초기 분석 범위는 04:50~05:50이었고 실제 장애는 05:22:01~05:22:38에 발생했습니다. 하지만 LLM이 조사 과정에서 분석 범위를 04:57~05:22로 잘못 좁히면서 장애 발생 1초 전에 탐색이 종료되고 있었습니다. 처음에는 조사 범위가 너무 좁은 것이 원인이라고 판단해 범위를 2.26배 넓혀 재실행했지만, 컨텍스트는 72%, 비용은 39% 증가한 반면 성능은 개선되지 않았습니다.

원인

- 장애 신호를 충분히 확보하지 못했습니다. 실제 장애가 예러가 아닌 23 초 지연으로 발생해 기존 status=error 중심의 Trace 탐색에서는 검색되지 않았고, Metric 도 5 분 단위 샘플링으로 인해 짧은 장애가 누락됐습니다.
- 분석 대상과 범위를 LLM 이 추정했습니다. 탐색 결과에서 하나의 범위와 하나의 traceId 만 선택하고, 이를 기준으로 LLM 이 분석 범위까지 결정했습니다. 처음 선택한 후보가 잘못되거나 범위가 장애 직전에 닫혀도 다른 후보를 확인하거나 범위를 보정할 장치가 없었습니다.

해결과 결과

문제를 확인한 뒤 조사 범위를 단순히 넓히는 대신, 탐색 방식 자체를 바꿨습니다. LLM이 처음부터 장애 구간

을 찾아야 했던 방식을, 코드가 만든 장애 사건 후보 중에서 선택하는 방식으로 변경했습니다. 지연·에러 Trace, 임계를 벗어나거나 값이 변한 Metric 샘플, 에러 로그 급증처럼 장애를 가리키는 Signal을 코드가 정해진 규칙으로 추출하고, 그중 같은 Trace 흐름에서 발생하면서 시간이 겹치거나 인접한 신호는 하나의 사건(Incident)으로 묶어 후보로 만들도록 처리했습니다. 분석 범위도 LLM이 임의로 추정하지 않고, 선택된 사건에 포함된 Signal의 시간 범위를 합친 뒤 Metric 샘플 간격만큼 여유를 더해 계산하도록 구현했습니다. 먼저 장애 신호가 충분히 수집되지 않던 Trace와 Metric의 탐색 조건을 보완했습니다.

- **Trace** - 기존 status=error 검색에 duration 기준의 지연 Trace 검색을 추가했습니다. 실제 장애 구간에서 해당 조건으로 Trace가 검색되는 것을 확인한 뒤 적용했습니다.
- **Metric** - 5분 단위 순간값만 확인하던 방식에서 min_over_time / max_over_time 을 사용하도록 변경해, 구간 내 짧게 발생한 이상도 탐색 결과에 남도록 했습니다.

변경 전후의 탐색 흐름은 다음과 같습니다.

기존 : 관측 데이터 → 탐색 결과 → LLM이 분석 범위 추정 → 분석
 변경 : Signal → Incident → LLM의 후보 선택 → 분석 범위 → 분석

장애가 발생한 구간이 사건 후보로 보존되기 때문에 탐색 누락이 줄고, LLM은 여러 후보의 근거를 비교해 분석 대상을 선택하는 판단에 집중하도록 했습니다. 첫 후보가 틀려도 다른 후보를 다시 확인할 수 있는 구조로 만들었습니다.

에러 Trace가 존재하지 않았던 동일 장애에서 정답에 해당하는 지연 Trace를 실제로 탐색할 수 있게 되었고, 해당 문항의 점수는 4점 → 85점으로 개선되었습니다. 이후 추가 개선을 거쳐 100점을 기록했습니다.

3) 관측 데이터를 원인 분석에 맞게 재구성하다

문제

장애 원인을 찾지 못한 사례가 여전히 있었습니다. 실행 데이터를 역추적한 결과, LLM의 추론이 아니라 관측 데이터가 분석에 필요한 형태를 잃는 것이 문제였습니다.

원인

- **Trace** - 장애 원인을 분석하려면 어떤 서비스가 어떤 서비스를 호출했는지부터 파악해야 했습니다. MSA 환경에서는 하나의 요청이 여러 서비스를 거치며 여러 span이 생성되는데, 기존에는 호출 관계를 별도로 구성하지 않고 span을 그대로 전달했습니다. 그 결과 LLM이 span의 서비스 정보와 부모 관계 등을 직접 연결해 서비스 간 호출 관계를 파악해야 하는 구조였고, 지연이나 실패가 어느 호출에서 발생했는지도 함께 판단해야 했습니다. 이 때문에 호출 관계를 해석하는 데 토큰이 소모되고, 관계를 잘못 연결하면 원인 분석의 정확도가 떨어질 수 있었습니다.
- **Log** - 팔로우 목록 NPE의 실제 원인은 FollowCondition.java:25였지만, 기존 LogQL은 ERROR/WARN 헤더 중심으로 조회해 예외 클래스와 파일·라인 정보가 분석 컨텍스트에서 빠지고 있었습니다. 또한 시간 범위의 ERROR/WARN 조회와 선택된 traceId의 전체 로그 조회, 두 경로가 동시에 존재해 같은 로그가 양쪽에 들어왔습니다. 실측 결과 4줄이 중복됐고, traceId 조회 결과의 약 27%(1.6KB)가 중복되었습니다.

해결과 결과

- **Trace · 호출 관계 복원** - 호출 관계를 먼저 복원해 서비스 간 그래프로 구성하도록 변경했습니다. 실제 장애에서 호출 대상 서비스가 다운된 경우에도 호출한 쪽 WebClient에 대상 서비스 정보가 남는 것을 확인해 호출 관계 판별 규칙을 보완했습니다. 이후 여러 span에 흩어진 정보를 A 서비스 → B 서비스 → 지연/실패 형태로 요약해 LLM에 전달하고, 필요한 경우에만 원본 span을 확인하도록 변경했습니다.
- **Log · 예외 정보 보강** - 분석용 조회에만 Exception, Caused by, 파일·라인 패턴을 추가해 실제 예외 발생 위치까지 전달했습니다. 탐색 단계는 발생량과 시간적 변화를 비교해 장애 후보를 찾는 역할이므로 기존 집계 기준을 유지해, 같은 로그라도 탐색과 분석의 목적에 따라 정보 수준을 다르게 구성했습니다.

- **Log · 중복 제거** - 한쪽 조회를 제거하면 `traceld` 가 없는 장애 로그나 요청 흐름까지 읽을 수 있었기 때문에 수집 경로 자체는 유지하고, 최종 조립 단계에서 (`timestamp, line`)이 동일한 로그만 제거했습니다.
- 누락됐던 예외 발생 위치가 분석 컨텍스트에 도달하면서, 정답 신호가 로그 스택에만 있던 문항은 **53점 → 88점**으로 개선됐습니다.
- 그 결과 RCA 파이프라인을 단순한 **원본 관측 데이터 → LLM 분석에서 관측 데이터 실측·검증 → 호출 관계 구조화 → 중복 제거·필요 정보 보장 → LLM 판단**의 구조로 변경했습니다. LLM의 추론력을 높이기 전에, LLM이 판단해야 할 데이터를 사람이 먼저 검증 가능한 **Evidence** 형태로 만드는 것부터 해결했습니다.

4) 성능 · 비용 최적화 : 컨텍스트를 줄이되, 근거는 버리지 않았다

문제

초기에는 하나의 LLM이 자연어 질문을 바탕으로 장애 시점과 분석 범위를 함께 판단하도록 구성했습니다. 정확한 장애 구간을 안정적으로 좁히기 어려워 넓은 범위의 로그 · 트레이스가 분석 단계까지 전달되는 문제가 있었고, 2)에서 탐색과 분석을 분리해 장애 사건을 먼저 찾고 해당 구간만 분석하도록 구조를 변경했습니다.

하지만 범위를 장애 사건 중심으로 좁힌 뒤에도 입력 토큰의 **98%**가 분석 단계에 집중되어 있었고, 그 대부분을 로그와 트레이스가 차지했습니다. 즉, **분석 범위를 좁히는 것만으로는 컨텍스트 크기를 충분히 줄일 수 없었습니다.**

원인

처음에는 로그가 장애 구간보다 넓게 수집되고 있다고 판단해 조회 범위를 **17.6%** 좁혔지만, 로그 크기는 사실 상 줄지 않았습니다. 조회 범위만 줄이는 방식으로는 한계가 있다고 판단해, 분석에 들어간 정보 중 실제 원인 판단에 사용된 부분을 확인했습니다.

이에 실제 분석 컨텍스트에 들어가는 로그와 기존 RCA 리포트에서 사용된 근거를 대조해, 어떤 정보가 실제 원인 판단에 사용되는지 분석했습니다. 그 결과 스택 트레이스의 라이브러리 내부 호출 줄은 전체 바이트의 **71%**를 차지하면서도 인용된 사례가 **0회**였고, 실제 서비스 코드를 가리키는 줄은 **3.7%**에 불과했지만 **32회** 원인 근거로 인용됐습니다. 즉, **토큰을 많이 차지하는 정보와 실제 RCA에 사용되는 정보가 일치하지 않았습니다.**

해결과 결과

먼저 탐색 단계에서 불필요한 원본부터 줄였습니다. 초기에는 탐색 LLM이 후보를 고르는 근거로 집계 원본 JSON까지 전달했는데, 이 데이터가 탐색 컨텍스트의 **94%**를 차지했습니다. 같은 탐색 데이터를 원본 포함 · 제외 두 조건으로 재실행해 비교한 결과, 원본이 없어도 후보 선택과 제외 결과가 동일하게 나오는 것을 확인했습니다. 이후 요약 정보만 전달하도록 변경해 **탐색 컨텍스트를 82.8% 절감**했습니다. 이렇게 탐색 단계를 줄인 뒤에도 전체 입력 토큰의 **98%**는 분석 단계에서 사용되고 있었습니다.

분석 단계에서는 **로그 전체를 잘라내는 대신, 원인 판단에 사용되지 않는 반복 정보만 접는 방식**으로 정제했습니다. 서비스 코드 · 예외 메시지 · 수치는 그대로 유지하고, 반복되는 라이브러리 내부 호출은 **... N frames** 형태로 접었습니다.

기존	변경 (접기 적용)
<pre>at com.example...FollowCondition.java:25 at com.example...FollowRepository.java:35 at org.springframework... at org.springframework... at reactor.core... ...</pre>	<pre>at com.example...FollowCondition.java:25 at com.example...FollowRepository.java:35 ... 4 frames (org.springframework, reactor.core)</pre>

완전히 동일한 로그가 반복되는 경우에는 한 건만 원문으로 남기고 발생 횟수 · 시간 범위 · 평균 간격을 통계로 제공했습니다. 숫자만 다른 근사 반복은 수치 자체가 근거가 될 수 있어 첫 발생과 마지막 발생은 원문 그대로 보존했습니다.

저장된 Loki 조회 응답 112건에 적용한 결과, 로그 컨텍스트는 **57.0% 감소**했습니다. 스택 트레이스가 많은 응답은 최대 약 83%까지 줄었고, 스택이 없는 응답은 크기를 그대로 유지했습니다.

트레이스는 OTLP의 중첩 구조를 RCA에 필요한 필드 중심의 단순한 key-value 구조로 변환하고, 여러 span에서 동일하게 반복되는 속성은 공통 블록에 한 번만 표시했습니다. span 자체와 시작 시각·소요 시간·장애 관련 속성은 그대로 유지했습니다. 저장된 트레이스 238건, 1,992개 span에 적용한 결과 **30.0% 감소**했습니다.

마지막으로 2)에서 정한 분석 범위를 채널별 특성에 맞게 적용했습니다. Trace·Log는 장애 구간에 집중하고, Metric은 복구 흐름과 복합 요인 확인을 위해 넓게 유지했습니다.

개선	결과	검증
로그 컨텍스트	57.0% 감소	원본 112건 검증
트레이스 컨텍스트	30.0% 감소	트레이스 238건 검증

정민채 · jmcabc0322@gmail.com · 상세 기록: docs/·reports/raw/ (본 문서의 모든 수치는 이 기록과 대조했습니다)